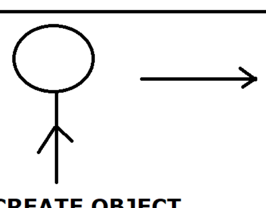
WAP to show how to work with static & non static field :
----

class Customer
{

}

BLC CLASS



CREATE OBJECT
IN ELC CLASS

Object Properties + Object Behavious
NSF NSM

WRITE OBJECT PROPERTIES AND
BEHAVIOUR IN BLC CLASS

Up to here how many ways we can initialize the non static fields :
----
3 ways :

1) By using reference variable [raj.rollNumber = 111]
2) By using method parameter [rollNumber = roll]
3) Initialization at the time of declaration [String name = "Scott"]

==
Create an Object for Customer

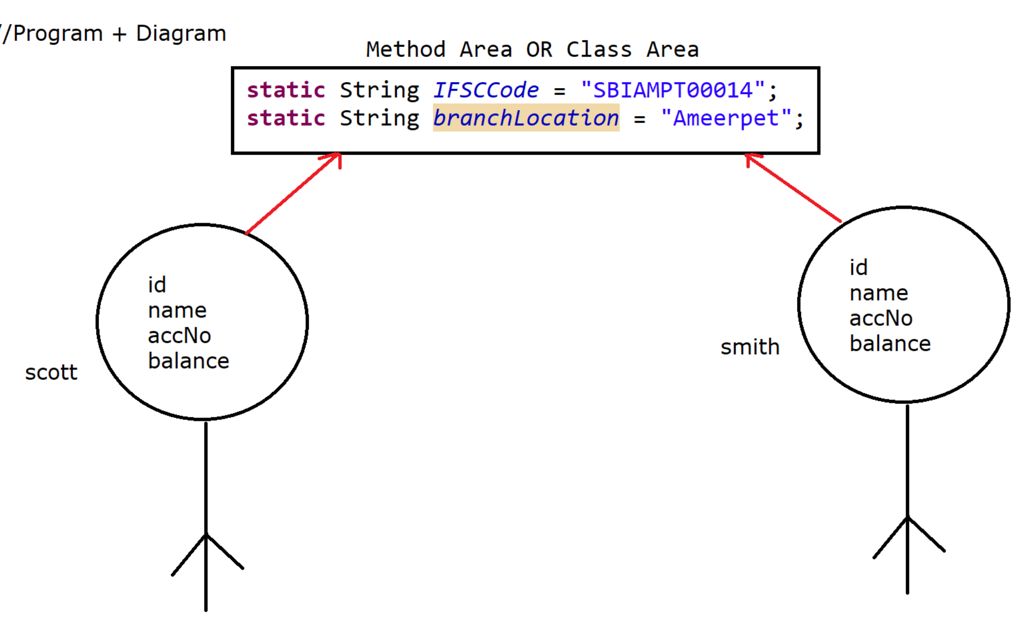
```
BLC = class Customer
      {
      }

ELC = Main OR CustomerMain OR CustomerDemo OR ELC

      class CustomerDemo
      {
      }

      FileName = CustomerDemo.java
```

//Program + Diagram




```java
package com.ravi.oop_28;

//ELC
public class CustomerDemo
{
    public static void main(String[] args)
    {
        Customer scott = new Customer();
        int id = Integer.parseInt(IO.readln("Enter customer id : "));
        String name = IO.readln("Enter Customer Name : ");
        long accNo = Long.parseLong(IO.readln("Enter account number : "));
        double balance = Double.parseDouble(IO.readln("Enter your balance amount : "));

        scott.setCustomerData(id,name, accNo,balance);
        scott.showCustomerData();

        IO.println("........................");
        Customer smith = new Customer();
        id = Integer.parseInt(IO.readln("Enter customer id : "));
        name = IO.readln("Enter Customer Name : ");
        accNo = Long.parseLong(IO.readln("Enter account number : "));
        balance = Double.parseDouble(IO.readln("Enter your balance amount : "));
        smith.setCustomerData(id, name, accNo, balance);
        smith.showCustomerData();
    }
}


class Customer
{
    int customerId;
    String customerName;
    long accountNumber;
    double accountBalance;

    static String IFSCCode = "SBIAMPT00014";
    static String branchLocation = "S R Nagar";

    public void setCustomerData(int id, String name, long accNo, double balance)
    {
        customerId = id;
        customerName = name;
        accountNumber = accNo;
        accountBalance = balance;
    }

    public void showCustomerData()
    {
        IO.println("Customer Id is :"+customerId);
        IO.println("Customer Name is :"+customerName);
        IO.println("Customer Account No is :"+accountNumber);
        IO.println("Customer Balance is :"+accountBalance);
        IO.println("IFSC Code is :"+IFSCCode);
        IO.println("Branch Location is :"+branchLocation);
    }

}
```

****** What is Data Hiding ?**

Data hiding is nothing but declaring our non-static fields with **private access modifier** so our data will not be **accessible** from outer world (Outside of BLC class) that means no one can access our data directly from outside of the class.

*We should provide the accessibility of our data through public **methods** so we can perform VALIDATION ON DATA which are coming from outer world.

```java
package com.ravi.data_hiding;

public class DataHiding
{
    public static void main(String[] args)
    {
          Customer john = new Customer();
          IO.println("Current Balance is :"+john.getCurrentBalance());
          john.deposit(5000);
          john.withdraw(2000);
    }
}

class Customer
{
    private double accountBalance = 10000; //Data hiding

    public void deposit(double amount)
    {
        //Data Validation
        if(amount <=0)
        {
            System.err.println("Sorry!! Amount cannot be deposited");
            return;
        }

        accountBalance += amount;
        IO.println("Current Balance after deposit :"+amount+" is :"+getCurrentBalance());
    }


    public void withdraw(double amount)
    {
        if(amount>accountBalance)
        {
            System.err.println("Insufficient Balance, Withdraw is not possible");
            return;
        }

        accountBalance -= amount;
        IO.println("Current Balance after withdraw :"+amount+" is :"+getCurrentBalance());
    }

    public double getCurrentBalance()
    {
        return accountBalance;
    }

}
```

Access modifiers in Java :

ELC class must be in the top & must be file name